PROJECT DOCUMENTATION

CodeGenie

AI-Powered Code Generator & Explainer

Submitted By:

Sukrutha Bodapally

ABSTRACT

CodeGenie is an intelligent web-based application that automates code generation and explanation using Large Language Models accessed through the Hugging Face Inference API. Users submit natural language prompts and receive complete, runnable code in Python, JavaScript, Java, C++, or C, accompanied by clear plain-English explanations of the generated output. Built on a Flask backend with SQLite, the system supports user authentication with bcrypt password hashing, snippet storage, activity logging, and structured feedback collection. The AI pipeline uses a cascading fallback strategy with Qwen2.5-Coder-32B-Instruct as the primary model, ensuring high availability during rate-limit events. The responsive frontend features syntax highlighting via Prism.js, a personal snippet dashboard, and a star-rating and chip-tag feedback modal. Users earn a verified badge after twelve feedback submissions. A Chart.js-powered admin dashboard provides visual insights into usage trends, language distribution, and feedback patterns, with role-based access maintaining strict separation between the user interface and admin panel.

CHAPTER 1: INTRODUCTION

1.1 Artificial Intelligence

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines programmed to think, learn, and solve problems autonomously. Over the past decade, AI has evolved from rule-based systems to deep learning models capable of understanding context, generating content, and making complex decisions. In software development, AI has enabled tools that automate repetitive tasks, detect bugs, suggest completions, and generate entire programs from a single sentence — fundamentally changing how developers write code.

1.2 Natural Language Processing and Code Generation

Natural Language Processing (NLP) is a branch of AI that enables computers to understand and generate human language. Modern transformer-based architectures, introduced in "Attention Is All You Need" by Vaswani et al. (2017), form the backbone of today's Large Language Models. These models are trained on massive corpora of text and code, learning statistical relationships between natural language descriptions and programming constructs. Instruction-tuned variants, refined through human feedback, can interpret a plain-English request and produce correct, executable code across multiple languages with remarkable accuracy.

1.3 Project Overview: CodeGenie

CodeGenie directly applies AI and NLP advances to make programming accessible to everyone. The application accepts a natural language prompt — exactly the kind of input LLMs are optimised to understand — and routes it to Qwen2.5-Coder-32B-Instruct through the Hugging Face Inference API. The model uses its understanding of both natural language and code syntax to return a complete, runnable implementation alongside a plain-English explanation. Built on Flask and SQLite, CodeGenie supports five programming languages — Python, JavaScript, Java, C++, and C — with Prism.js syntax highlighting making AI-generated output immediately readable.

Users can save snippets to a personal dashboard and provide structured feedback through star ratings and quick-pick chip tags on sign-out. A verified badge rewards consistent participation after twelve submissions. An admin analytics dashboard built with Chart.js provides administrators visibility into language trends, usage patterns, and feedback analysis —

delivering a complete, deployment-ready application that bridges natural language intent and working code.

1.4 Objectives and Scope

The primary objectives of CodeGenie are: to provide fast, accurate, multi-language code generation from natural language prompts; to make generated code immediately readable through syntax highlighting; to collect structured user feedback for continuous improvement; and to give administrators full visibility into usage patterns through an analytics dashboard.

The scope of the project encompasses user registration and authentication, code generation for Python, JavaScript, Java, C++, and C, snippet persistence and retrieval, a feedback modal with star ratings and chip tags, a verified user badge system, an admin panel with four Chart.js charts (activity bar, language usage, rating doughnut, feedback tags), and deployment-ready configuration with environment variable management. Out of scope for the current version are real-time collaborative coding, execution of generated code, and OAuth-based social login.

CHAPTER 2 SYSTEM STUDY

2.1 Problem Statement

Writing code from scratch is time-consuming, error-prone, and inaccessible to non-experts. Existing code generators often require API keys managed by the end user, lack a persistent storage layer, provide no explanation of the generated output, and offer no administrative visibility into usage. CodeGenie addresses all of these gaps in a single, self-contained web application.

2.2 Features of the System

2.2.1 User Authentication

Users register with a name, email, and password. Passwords are hashed using Werkzeug's bcrypt-compatible generate_password_hash before storage — plain-text credentials are never persisted. Sessions are managed server-side via Flask's signed cookie mechanism using a secret key loaded from the .env file. A default demo account and a seeded admin account are created automatically on first run.

2.2.2 Multi-Language Code Generation

The homepage presents a language selector bar with five options: Python, JavaScript, Java, C++, and C. The selected language is passed with the user's natural language prompt to the Flask backend, which embeds it into the system and user messages sent to the Hugging Face Inference API. The AI model returns a structured response in CODE: / EXPLANATION: format. A cascading fallback mechanism ensures continuity if the primary model (Qwen2.5-Coder-32B) is unavailable, falling back to Qwen2.5-Coder-7B and then Mistral-7B.

2.2.3 Syntax Highlighting

Generated code is rendered inside a <pre><code> element. Prism.js — loaded from CDN — applies language-specific tokenisation and colour-coding in the browser without any server involvement. Separate grammar packs are loaded for Python, JavaScript, Java, C, and C++.

2.2.4 Explanation Panel

The AI response is parsed to extract the EXPLANATION: section, which is cleaned of prefix artifacts and displayed in a dedicated panel below the prompt area. This gives users, especially beginners, an immediate plain-English understanding of the generated code without needing to read every line.

2.2.5 Snippet Dashboard

Authenticated users can save any generated snippet (prompt + code + language) to their personal dashboard. The dashboard displays all saved snippets with expandable code boxes, local timestamps, and language badges. A Home navigation button allows quick return to the generation interface.

2.2.6 Feedback System

When a regular user signs out, a feedback modal is displayed before the logout completes. The modal presents a 5-star rating and six quick-pick chip tags (Great code quality, Fast generation, Good explanation, UI is clean, Needs improvement, Wrong output). Chips are optional; at least one star is required to submit. Admin users and verified users bypass the modal. Feedback is stored in the feedback table and is accessible through the admin panel.

2.2.7 Verified User Badge

Users who submit 12 or more feedback entries earn a verified badge, displayed as an orange checkmark on their avatar in the homepage header. A celebration popup is shown at the moment of verification. Verified users skip the feedback modal on all subsequent sign-outs. The badge status is synced from the server on every page load via the /api/me endpoint, ensuring it appears immediately even if the user achieved verified status in a previous session.

2.2.8 Admin Dashboard

The /admin route, accessible only to users with is_admin=True, provides a comprehensive analytics dashboard built with Chart.js. It displays five stat cards (Total Users, Snippets Saved, Avg Rating, Top Language, Verified Users), a 14-day activity bar chart, a language usage bar chart, a rating distribution doughnut chart, a feedback tags horizontal bar chart, a full registered users table with last-active indicators and per-user metrics, and a scrollable list of all feedback submissions.

2.3 Advantages of the System

- No server-side code execution — all syntax highlighting is client-side, keeping the backend lightweight and secure
- Cascading model fallback ensures the system remains functional even during Hugging Face API rate-limit events

- Structured feedback collection (stars + chips) provides quantitative and qualitative insight without requiring users to type
- Role-based access control ensures strict separation between the user interface and the admin panel
- SQLite with SQLAlchemy ORM allows zero-configuration deployment while remaining fully replaceable with PostgreSQL for production scale
- Environment variable management via python-dotenv keeps API tokens out of source code
- The verified badge system gamifies engagement and reduces feedback fatigue for long-term users

2.4 Feasibility

2.4.1 Technical Feasibility

The system is built entirely on mature, well-documented open-source technologies. Flask, SQLAlchemy, and Werkzeug are production-proven Python libraries with large communities. The Hugging Face Inference API is free at moderate usage levels and supports the Qwen Coder model family. Prism.js and Chart.js are lightweight CDN-hosted libraries with broad browser compatibility. The entire application runs on a single Python process, making it deployable on any platform that supports Python 3.9+ including Railway, Render, and Fly.io.

2.4.2 Economic Feasibility

All core dependencies are free and open-source. The Hugging Face Inference API is available at no cost for non-commercial use at standard rate limits. SQLite requires no separate database server, eliminating infrastructure cost. The application is deployed on PythonAnywhere's free tier, which provides always-on hosting sufficient for demonstration and academic use at zero cost.

2.4.3 Operational Feasibility

The application is designed for simplicity of operation. Setup requires only Python installation, a virtual environment, and a single .env file containing two values. The database initialises automatically on first run. No manual migration scripts are required. A SETUP.md file in the repository provides step-by-step instructions for any user to run the application locally within five minutes

CHAPTER 3 SYSTEM DESIGN

3.1 Overall System Architecture

CodeGenie follows a three-tier client-server architecture. The presentation tier consists of four HTML/CSS/JavaScript pages (login.html, homepage.html, dashboard.html, admin.html) served directly by Flask. The application tier is the Flask app.py module, which handles routing, authentication, session management, database operations, and AI API calls. The data tier consists of the SQLite database codegenie.db, which stores the users, snippets, activity_logs, and feedback tables.

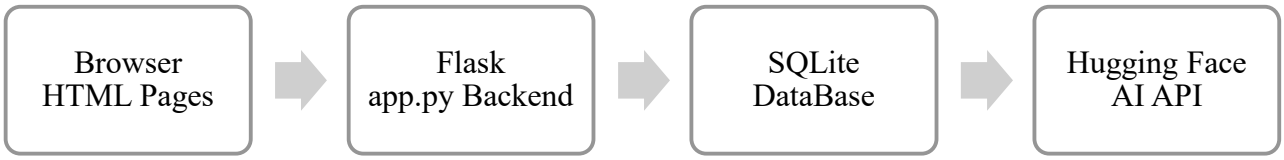


Figure 3.1: Overall System Architecture

3.2 Database Schema

The database consists of four tables with the following structure:

Table	Key Columns	Purpose
users	id, name, email, password, created_at, last_active, is_admin, feedback_count, is_verified	User accounts and authentication
snippets	id, user_email, prompt, code, language, created_at	Saved code snippets per user
activity_logs	id, user_email, action, language, created_at	Audit trail of all user actions
feedback	id, user_email, user_name, rating, message, created_at	Star ratings and chip-tag submissions

Figure 3.2: Database Schema

3.3 User and Admin Flow Diagram

The flow below shows the complete journey of a regular user from login to logout, and the separate path taken by an admin user.

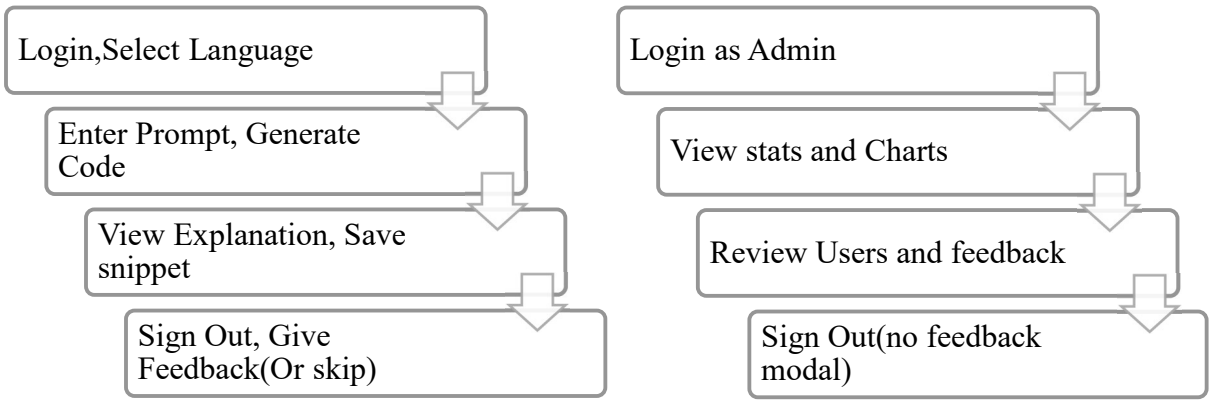


Figure 3.3: User and Admin flow

CHAPTER 4: METHODOLOGY

4.1 Development Methodology

CodeGenie was developed using an Iterative Incremental methodology, dividing the project into clearly defined development phases, each building directly on the foundation of the previous. This approach allows for early delivery of core functionality, continuous integration of user feedback, and controlled scope expansion at each stage. The system was developed in focused phases — beginning with core infrastructure and authentication, progressing through enhanced user experience features, and culminating in administrative analytics and user engagement capabilities. Each phase was scoped to a specific set of user-facing features and technical objectives, fully tested end-to-end before the next phase began.

4.2 Technology Stack

Layer	Technology	Version	Role
Backend	Flask	3.0.0	Web framework, routing, session management
Backend	Flask-SQLAlchemy	3.1.1	ORM for SQLite database interactions
Backend	Werkzeug	3.0.1	Password hashing (bcrypt-compatible)
Backend	python-dotenv	1.0.0	Environment variable management from .env
Backend	requests	2.31.0	HTTP calls to Hugging Face Inference API
Database	SQLite	Built-in	Serverless persistent relational database
AI Model	Qwen2.5-Coder-32B	Primary	Primary code generation model via HF Router
AI Model	Qwen2.5-Coder-7B	Fallback	Secondary model if primary is rate-limited
Frontend	Prism.js	CDN	Client-side syntax highlighting for 5 languages
Frontend	Chart.js	4.4.1	Admin dashboard charts (bar, doughnut, line)
Frontend	Plus Jakarta Sans	CDN	UI typography for main interface
Frontend	JetBrains Mono	CDN	Monospace font for code display

Figure 4.1: Technology Stack

4.3 Interfacing the Hugging Face Model

The primary model is **Qwen2.5-Coder-7B-Instruct** with **Qwen2.5-Coder-32B-Instruct** as fallback — both developed by Alibaba Cloud and specifically fine-tuned for code generation across multiple programming languages.

A system message establishes the structured output contract — the model is instructed to respond strictly in CODE: followed by EXPLANATION: format, with no markdown code blocks. The user message embeds the selected programming language label and the user's natural language prompt. Additional parameters — temperature: 0.2 and top_p: 0.95 — keep the output focused and deterministic rather than creative.

CODE:

```
# Cascading model fallback (app.py)

models = [
    "Qwen/Qwen2.5-Coder-7B-Instruct",    # Primary
    "Qwen/Qwen2.5-Coder-32B-Instruct",   # Fallback
]
for model in models:
    url = "https://router.huggingface.co/v1/chat/completions"
    payload = {
        "model": model,
        "messages": [
            {"role": "system", "content": "You are CodeGenie.Reply in
CODE:/EXPLANATION: format only"},
            {"role": "user",   "content": f"Write complete, working
{lang_label} code for: {prompt}"},
        ],
        "max_tokens": 600,
        "temperature": 0.2,
        "top_p": 0.95,
    }
```

4.4 Security Methodology

- Passwords stored as bcrypt hashes — plain text never persisted
- Flask sessions signed with FLASK_SECRET_KEY loaded from .env — not hardcoded

- HUGGINGFACE_API_TOKEN loaded from .env — not present in source code or repository
- .gitignore excludes .env, *.db, and virtual environment folders
- Admin routes protected by server-side is_admin session check — client-side flag alone is insufficient
- All SQL interactions via SQLAlchemy ORM — no raw SQL strings, eliminating SQL injection risk

CHAPTER 5: IMPLEMENTATION

5.1 Project Structure

```
CodeGenie/
├── templates/
│   ├── login.html
│   ├── homepage.html
│   ├── dashboard.html
│   └── admin.html
├── app.py
├── codegenie.db
├── requirements.txt
├── .env            # Secrets (not in repository)
├── .env.example    # Template for .env
└── .gitignore
```

Figure 5.1: Project Directory Structure

5.2 User Authentication Implementation

User registration hashes the password before storing it. The login endpoint verifies the hash and creates a signed server-side session containing the user's email, name, is_admin flag, feedback_count, and is_verified status.

CODE:

```
# app.py - Registration
hashed_pw = generate_password_hash(password)
user = User(name=name, email=email, password=hashed_pw)
db.session.add(user)
db.session.commit()

# app.py - Login
user = User.query.filter_by(email=email).first()
if user and check_password_hash(user.password, password):
    session["user"] = {
        "email": user.email,
        "name": user.name,
        "is_admin": bool(user.is_admin),
        "is_verified": bool(user.is_verified),
        "feedback_count": user.feedback_count or 0
    }
    return jsonify({"success": True, "is_admin": bool(user.is_admin)})
```

5.3 Code Generation and Response Parsing

The generate endpoint receives the prompt and language, builds the structured messages array, calls the Hugging Face API with fallback logic, and then parses the response to extract the code and explanation sections separately.

CODE:

```
# app.py - Response parsing
def _parse_code_and_explanation(raw):
    if "explanation:" in raw.lower():
        idx  = raw.lower().index("explanation:")
        code = raw[:idx].replace("CODE:","").strip()
        expl = raw[idx+12:].strip()
    else:
        code, expl = raw.replace("CODE:","").strip(), ""
    # Clean markdown fences if model ignores instructions
    code = re.sub(r"```[a-z]*\n?", "", code).strip("`").strip()
    if expl.startswith(":"): expl = expl.lstrip(":").strip()
    return code, expl
```

5.4 Frontend: Language Switching and Syntax Highlighting

CODE:

```
// homepage.html - Language selection
var lang = "python";
document.querySelectorAll(".lbtn").forEach(function(b) {
    b.addEventListener("click", function() {
        lang = b.dataset.lang;
        document.querySelectorAll(".lbtn").forEach(x =>
x.classList.remove("on"));
        b.classList.add("on");
    });
});

// After code is received from API:
var co = document.getElementById("codeOut");
co.textContent = data.code;
co.className   = "ce language-" + (data.language || "plaintext");
if (window.Prism) Prism.highlightElement(co);
```

5.5 Feedback System and Verified Badge

```
# app.py - Feedback submission with verification logic
user.feedback_count = (user.feedback_count or 0) + 1
newly_verified = False
if user.feedback_count >= 12 and not user.is_verified:
    user.is_verified = True
    newly_verified   = True
db.session.commit()
```

```
return jsonify({
    "success":        True,
    "feedback_count": user.feedback_count,
    "is_verified":    bool(user.is_verified),
    "newly_verified": newly_verified
})
```

5.6 Admin Stats API

```
# app.py - Per-user accurate feedback counts
fb_counts = dict(
    db.session.query(Feedback.user_email, func.count(Feedback.id))
    .group_by(Feedback.user_email).all()
)
fb_ratings = dict(
    db.session.query(Feedback.user_email, func.avg(Feedback.rating))
    .group_by(Feedback.user_email).all()
)
# Each user entry in the response includes:
# "feedback_count": fb_counts.get(u.email, 0),
# "avg_feedback":   round(float(fb_ratings[u.email]),1) if u.email in
fb_ratings else None
```

5.7 Output Screenshots

The following figures show the key screens of the CodeGenie application.

Figure: Login and Registration Page

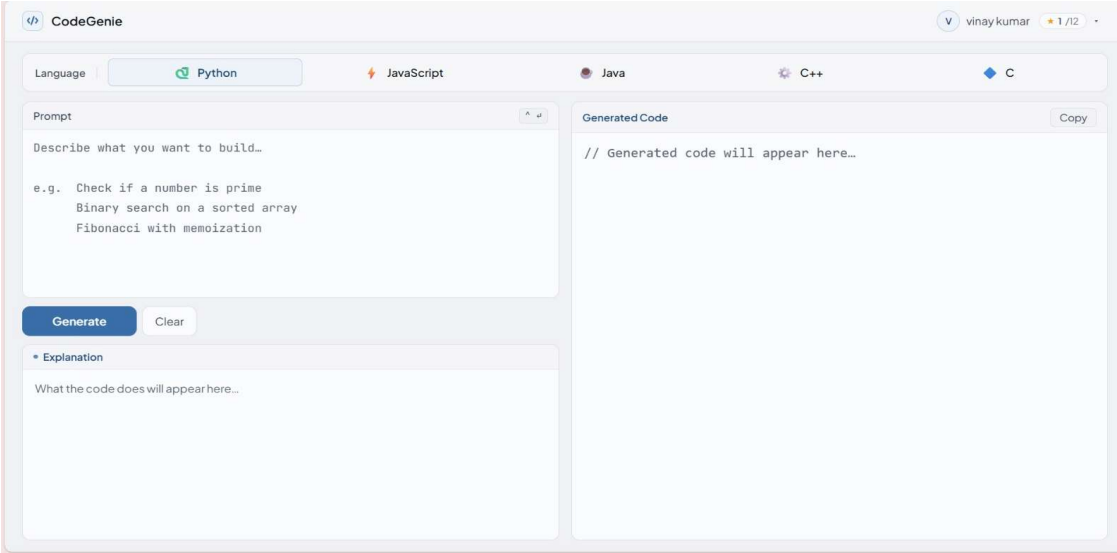


Figure: Homepage - Language Bar, Prompt Input, and Generate Button

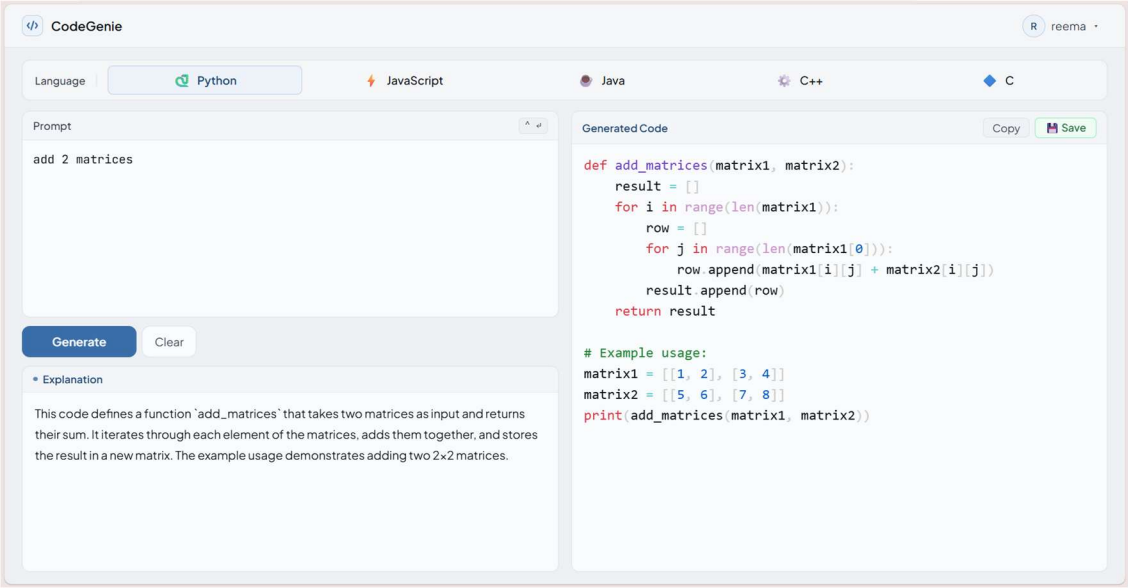


Figure: Homepage - Generated Code with Syntax Highlighting

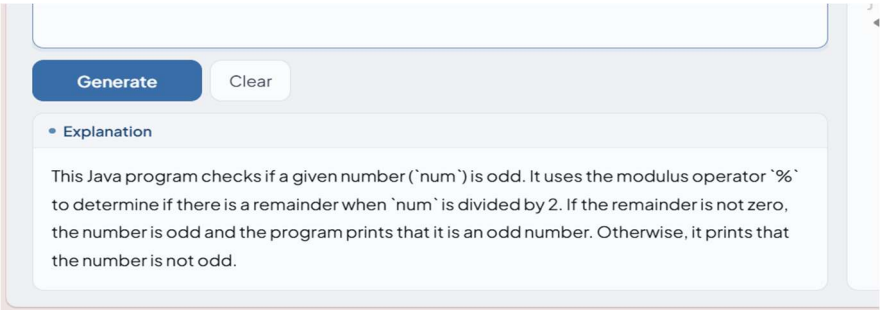


Figure: Homepage - Explanation Panel with AI Description

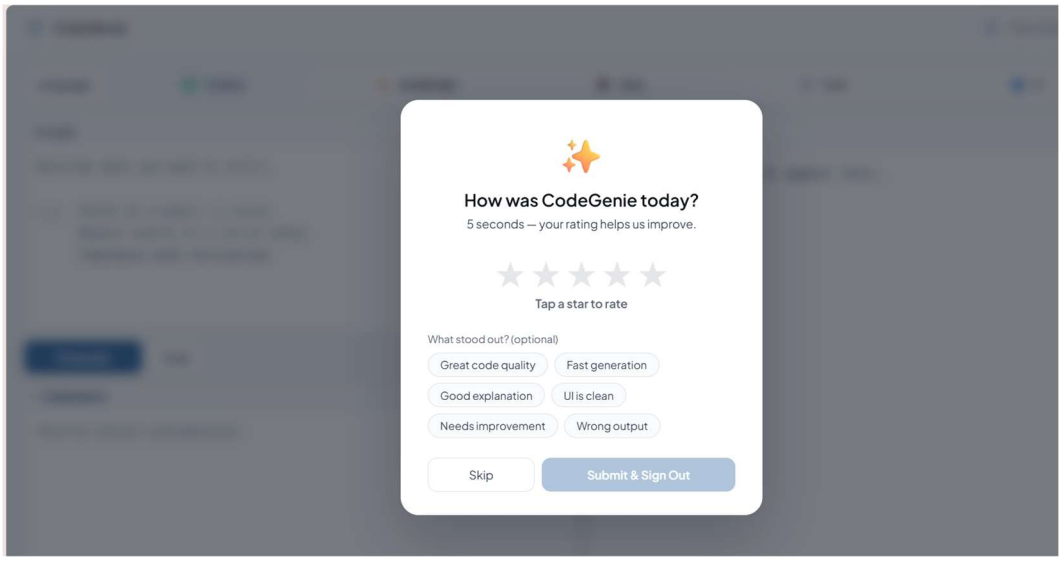


Figure: Feedback Modal - Star Rating and Chip Tags

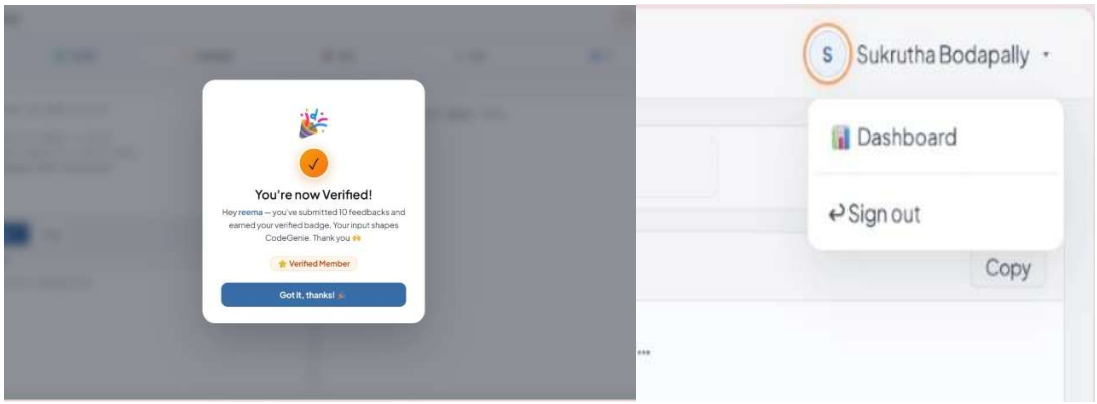


Figure: Verified User Badge on Avatar

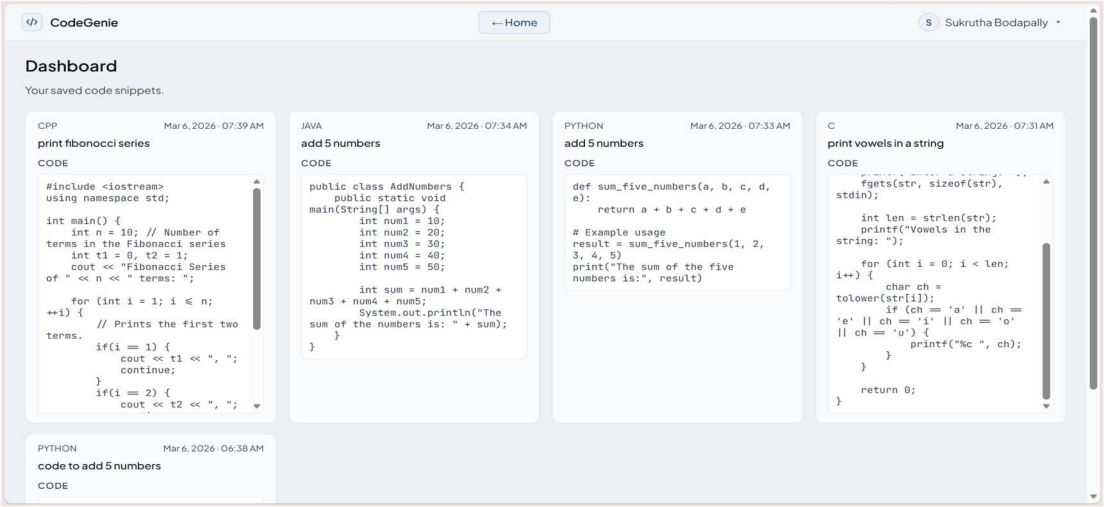
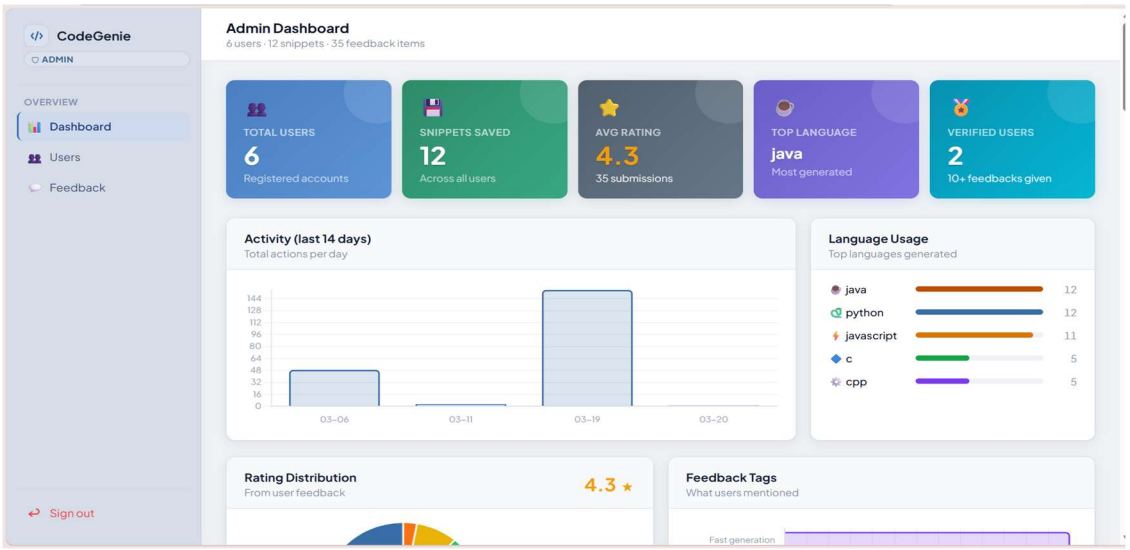


Figure: Snippet Dashboard - Saved Code Cards



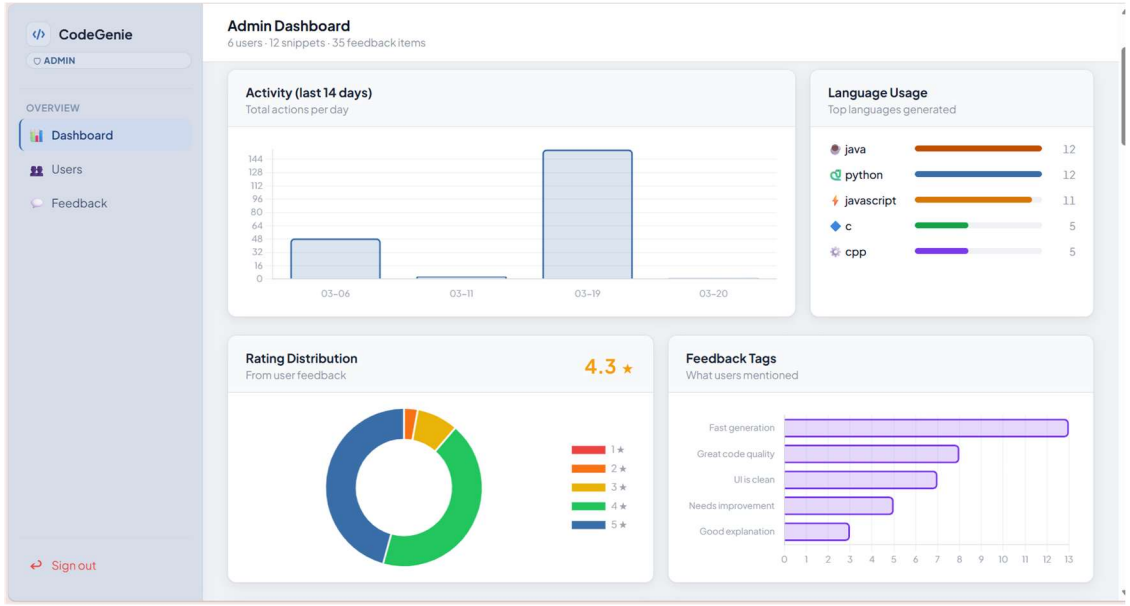


Figure: Admin Dashboard - Stat Cards and Charts

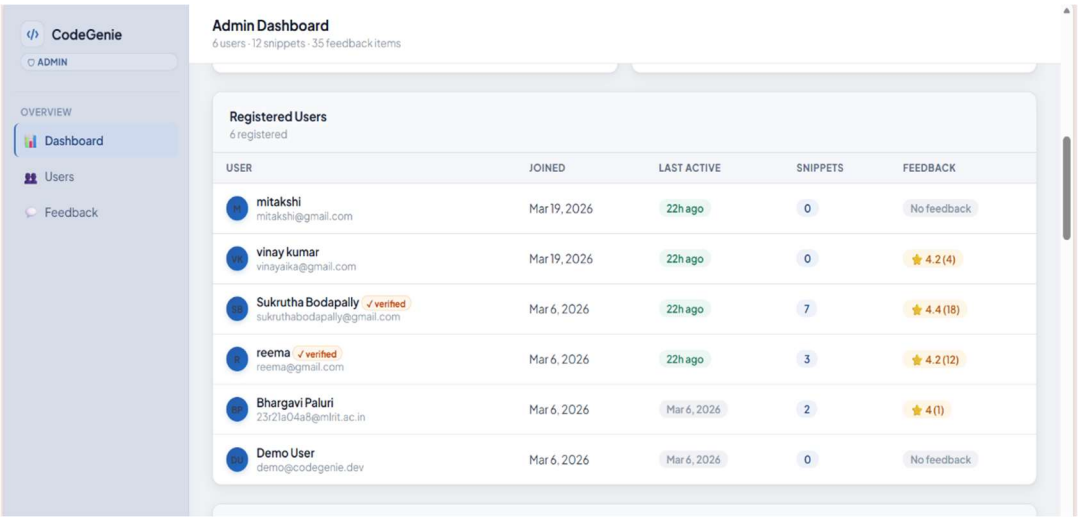


Figure: Admin Dashboard - Users Table with Verified Badge

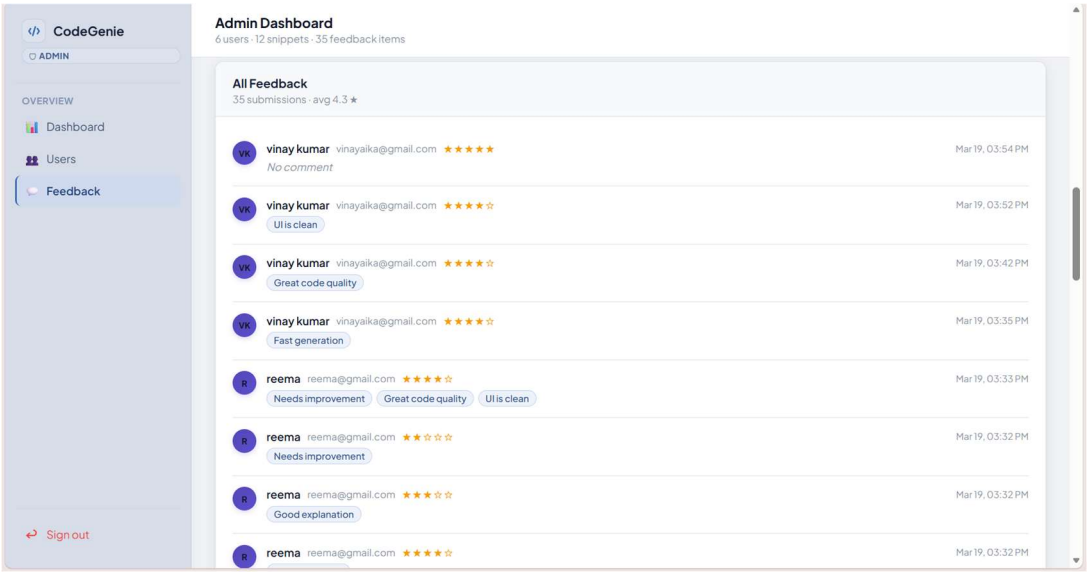


Figure: Admin Dashboard – Users Feedbacks and Comments

CHAPTER 6: CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

CodeGenie successfully demonstrates the practical application of Large Language Models in an accessible, full-stack web environment. By accepting natural language prompts and returning syntax-highlighted, explained code across five programming languages, the system bridges the gap between programming intent and working implementation — making software development approachable for beginners and efficient for experienced developers alike. Secure session-based authentication, multi-language code generation with cascading model fallback, a personal snippet dashboard, and structured feedback collection combine to deliver a cohesive, production-ready application.

The admin analytics panel powered by Chart.js provides administrators complete visibility into usage trends, language distribution, and user engagement, while role-based access ensures strict separation between the user and admin interfaces. The verified badge system gamifies participation and encourages consistent feedback. Overall, CodeGenie proves that open-source LLMs through the Hugging Face Inference API can deliver intelligent programming assistance without commercial dependency — putting the power of AI directly into users' hands.

6.2 Future Enhancements

- **Code Execution Sandbox:** Integrate a sandboxed execution engine such as Judge0 to allow users to run generated code directly in the browser without leaving the application.
- **Snippet Search and Filtering:** Add a search bar and language or date filters to the dashboard for efficient navigation as the snippet library grows over time.
- **OAuth Login:** Add Google and GitHub sign-in via trusted third-party providers, replacing the manual email and password flow for a smoother onboarding experience.
- **Multi-Model Selection:** Expose model choice to users, allowing selection between speed-optimised and quality-optimised models based on their immediate requirements.
- **Mobile Application:** Develop a React Native or Flutter mobile wrapper around the existing API to deliver a native mobile experience on Android and iOS.

6.3 References

- Chen, M., et al. (2021). *Evaluating Large Language Models Trained on Code.* arXiv:2107.03374.
- Qwen Team. (2024). *Qwen2.5-Coder Technical Report.* arXiv:2409.12186.
- Hugging Face. (2025). *Inference API Documentation.* https://huggingface.co/docs/api-inference
- Pallets Projects. (2024). *Flask Documentation (3.0.x).* https://flask.palletsprojects.com
- SQLAlchemy. (2024). *SQLAlchemy 2.0 Documentation.* https://docs.sqlalchemy.org
- Prism.js. (2024). *Lightweight Syntax Highlighter.* https://prismjs.com
- Chart.js. (2024). *Chart.js Documentation.* https://www.chartjs.org/docs